

製造ダッシュボード API — API仕様設計

003_DD-API — 003_DD を支える。ハンドラーは 003_DD-FN で設計。要件 REQ-028～REQ-042。

改版履歴

項目	値
ドキュメントID	003_DD-API
システム名	ProductionManagementAI
サブシステム名	製造指示（Production orders）
作業項目	WI-004
作成者	Claude（ThanhTN 向け）
作成日	2026-09-22
最終更新者	Claude（ThanhTN 向け）
最終更新日	2026-09-22

版	日付	作成者	改版内容
1	2026-09-22	Claude（ThanhTN 向け）	新規作成
2	2026-09-22	Claude（ThanhTN 向け）	<code>GET /api/system/health</code> を追加（DEC-017、DEC-019、DEC-020）。今週について <code>workload[].weekStart</code> を明確化
3	2026-09-23	Claude（ThanhTN 向け）	WI-005：UI を日本語化。例の製品名を日本語の自動車部品にした（WI-005 DEC-002）。引用する UI テキストを日本語で示す（WI-005 DEC-003）。項目・ルール・API・処理の変更はない

概要と操作の一覧

項目	値
API／モジュール名	製造ダッシュボード API
概要	SCR-003 の背後の読み取り専用の JSON エンドポイント 1 つで、1 つの一貫したスナップショットからすべてのウィジェットの数値を返す（DEC-011、DEC-015）。パラメータを受け取らず、何も書き込まない（REQ-039）

No	エンドポイント／メソッド	ハンドラー	目的	備考
1	<code>GET /api/dashboard</code>	<code>DashboardController.Get</code> → <code>DashboardService.GetSnapshotAsync</code>	ダッシュボードのスナップショット	WI-004 で新規
2	<code>GET /api/system/health</code>	<code>SystemController.Health</code> → <code>SystemHealthService.CheckAsync</code>	ヘルス表示のためのデータベースの到達可否	WI-004 で新規（DEC-017）

共通のルール。001_DD-API と 002_DD-API から変更なし：

- **認証**：Cookie セッション（0002_ADR）とポリシー `ProductionOrderEditor` = ロール `Admin` または `Operator`。セッションなし → 空のボディの 401。ロールのないセッション → 空のボディの 403。002_DD-API が挙げたのと同じ理由で既存のポリシーを再利用する：別の読み取り専用のポリシーは、WI-001 DEC-015 が決めていない権限の分割を意味してしまう。
- **コンテンツタイプ**： `application/json`。エラーは 001_DD-API の形の `application/problem+json`（RFC 9457）。スタックトレースや例外のテキストは決して返さない。
- **JSON**： camelCase。日付は YYYY-MM-DD（工場ローカル）、タイムスタンプは Z 付きの ISO 8601 UTC、列挙型は文字列。
- **CSRF**： 該当なし — GET、状態の変更なし、ボディなし（WI-002 DEC-020 の方針は変わらない）。
- **キャッシュ**： レスポンスは `Cache-Control: no-store` を持つ。数値はある時点のスナップショットなので、ブラウザやプロキシが後の / への訪問に以前のスナップショットで応答してはならない。

システム全体の API 登録簿の項目

ID	エンドポイント	概要	備考
API-DSH-01	GET /api/dashboard	製造ダッシュボードのスナップショット	新規。ダッシュボードのグループの最初の項目
API-SYS-01	GET /api/system/health	ログイン済みのユーザー向けのサーバーとデータベースの状態	新規。匿名の死活確認 /health は API 登録簿の項目ではなく、変更しない

API-PRD-01 と API-PO-01 ~ API-PO-04 は 001_DD-API と 002_DD-API に登録したとおり。

共通のレスポンスオブジェクト： `DashboardOrder`

`overdue.orders` または `dueSoon.orders` の 1 要素。意図的に狭い行：ダッシュボードはこれらの項目を表示し、どこにもリンクしないので（DEC-006）、`isOverdue`（グループがそう示す）、`updatedAt`、備考、バージョンはどれも持たない。

No	名前	変数名	型	長さ	必須	繰り返し（配列）	値の加工	例	説明	備考
1	ID	<code>id</code>	string (uuid)	36	yes	no	—	0197e4a0-...	<code>production_orders.id</code>	描画のキーとしてだけ使い、表示しない
2	指示番号	<code>orderNumber</code>	string	13	yes	no	—	P0-2026-00022		
3	製品	<code>product</code>	object	—	yes	no	<code>products</code> との結合	—	{ "id", "sku", "name" } — 002_DD-API と同じ <code>ProductSummary</code>	"{sku} — {name}" として描画（003_BD M-12）
4	数量	<code>quantity</code>	integer	—	yes	no	—	2390		
5	納期	<code>dueDate</code>	string (date)	10	yes	no	—	2026-09-12	工場ローカル	003_BD M-15
6	ステータス	<code>status</code>	string	—	yes	no	列挙型の名前	InProgress	ここでは常に <code>Draft</code> または <code>InProgress</code>	ラベルはクライアント側（M-11）

共通のレスポンスオブジェクト： `OrderGroup`

No	名前	変数名	型	必須	繰り返し（配列）	説明
1	合計	<code>total</code>	integer	yes	no	返したもののだけでなく、グループのすべての製造指示
2	製造指示	<code>orders</code>	<code>DashboardOrder[]</code>	yes	yes	最大 10 件、納期、次に指示番号の順（D-01、D-02）

共通のレスポンスオブジェクト： `CountAndQuantity`

No	名前	変数名	型	必須	説明
1	件数	<code>orderCount</code>	integer	yes	なければ 0
2	数量	<code>quantity</code>	integer (int64)	yes	<code>quantity</code> の合計。なければ 0。64 ビット：32 ビットの数量の合計は 2^{31} を超えるが、 2^{53} までは JavaScript の数値で正確に表せる

エンドポイント／メソッドの設計

1. GET /api/dashboard

項目	値
説明	SCR-003 のすべての数値を 1 つの一貫したスナップショットから計算し、スナップショット時刻と数値の計算の基準にした工場ローカルの日付とともに返す
戻り値の型	<code>DashboardResponse</code>
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数：なし。エンドポイントはルート、クエリ、ボディのパラメータをバインドしない。リクエストとともに送られたクエリ文字列は無視され、期間や件数を変えることはできない（003_BD §5）。したがって 400 のレスポンスはない。

処理概要：認可してから `DashboardService.GetSnapshotAsync`（003_DD-FN §1）に委譲する。それが時計を一度読み、工場ローカルの日付からすべての期間を導き（003_DD-FN §2）、003_DB の 7 つの文を 1 つの読み取り専用の `REPEATABLE READ` トランザクションで実行し（§3）、結果を整える（§4）。レスポンスは完全かエラーのどちらかで — 部分的になることはない（REQ-028）。

処理フロー

ステップ	説明	呼び出し
1	認可	ポリシー <code>ProductionOrderEditor</code>
2	スナップショットを組み立てる	<code>DashboardService.GetSnapshotAsync</code> → 003_DD-FN §1
3	<code>DashboardResponse</code> と <code>Cache-Control: no-store</code> 付きの 200	—

戻り値

型	名前	説明
<code>DashboardResponse</code>	ボディ	レスポンスの項目を参照

リクエストの項目：該当なし — エンドポイントは入力を受け取らない。

レスポンスの項目 — `DashboardResponse`

No	名前	変数名	型	長さ	必須	繰り返し（配列）	値の加工	例	説明	備考
1	スナップショット時刻	asOf	string (date-time)	—	yes	no	UTC	2026-09-22T05:05:12Z	時計を読んだ時点	工場時刻で表示（M-17）
2	工場の今日	today	string (date)	10	yes	no	T	2026-09-22	すべての数値の計算の基準にした日付	
3	工場のタイムゾーン	timeZone	string	—	yes	no	PlantOptions.TimeZone	Asia/Tokyo	クライアントが asOf を、ブラウザではなく数値と同じタイムゾーンで描画できるようにする	
4	ステータス別件数	statusCounts	object	—	yes	no	Q1	—	{ total, draft, inProgress, completed, cancelled }、整数。製造指示のないステータスは 0	REQ-029。total = 4 つの合計
5	納期遅れ	overdue	OrderGroup	—	yes	no	Q3a	—	D-01	REQ-030
6	納期間近	dueSoon	OrderGroup	—	yes	no	Q3b	—	D-02	REQ-030
7	作業量	workload	object[]	—	yes	yes（ちょうど 10）	Q2、0 で埋める	—	表示順の D-03 の区分	REQ-031、DEC-009
7.1	区分の種類	workload[].kind	string	—	yes	—	—	week	overdue week later	最初は overdue、次に 8 個の week、最後は later
7.2	週の開始	workload[].weekStart	string (date) null	10	yes	—	—	2026-09-28	区分が扱う最初の日付：その週の月曜日。ただし今週は最初の日付が today（それより前の日は納期遅れ、D-03）。overdue と later では null	
7.3	週の終了	workload[].weekEnd	string (date) null	10	yes	—	—	2026-10-04	その週の日曜日。overdue と later では null	対応する表が範囲を示す（M-16）
7.4	件数／数量	workload[].orderCount、.quantity	integer, integer (int64)	—	yes	—	—	7、3150		CountAndQuantity の項目
8	上位の製品	topProducts	object[]	—	yes	yes（0～10）	Q4	—	D-04、未完了数量の降順、次に SKU	REQ-032
8.1	製品	topProducts[].product	ProductSummary	—	yes	—	—	—		
8.2	未完了数量	topProducts[].openQuantity	integer (int64)	—	yes	—	—	2390		
8.3	未完了の製造指示数	topProducts[].activeOrderCount	integer	—	yes	—	—	2		
9	今週の完了	completedThisWeek	CountAndQuantity + from	—	yes	no	Q5	—	D-05。from = W(T)	REQ-034
10	今月の完了	completedThisMonth	CountAndQuantity + from	—	yes	no	Q5	—	D-06。from = T の月の 1 日	REQ-034

No	名前	変数名	型	長さ	必須	繰り返し（配列）	値の加工	例	説明	備考
11	期限内	onTime	object	—	yes	no	Q5	—	{ onTimeCount, completedCount, windowStart }。windowStart = T - 29。率は送らない：クライアントが導く（M-13）ので、丸めのルールは1つで、置き場所も1か所	REQ-035
12	リードタイム	leadTime	object	—	yes	no	Q5	—	{ averageDays, orderCount, windowStart }。averageDays はサーバーが小数1桁に四捨五入（0から遠ざかる方向）した平均、orderCount = 0 なら null	REQ-037
13	完了の推移	completionTrend	object[]	—	yes	yes（ちょうど12）	Q6、0で埋める	—	{ weekStart, weekEnd, orderCount }、古い順。最後が今週	REQ-036

from と windowStart の日付を送るので、対応する表とタイルのキャプションは、クライアントが工場の暦を再計算しなくても期間を正確に示せる。

レスポンスの例（抜粋）：

```

{
  "asOf": "2026-09-22T05:05:12Z",
  "today": "2026-09-22",
  "timeZone": "Asia/Tokyo",
  "statusCounts": { "total": 124, "draft": 35, "inProgress": 25, "completed": 56, "cancelled": 8 },
  "overdue": {
    "total": 15,
    "orders": [
      {
        "id": "0197e4a0-0000-7000-8001-000000000078",
        "orderNumber": "PO-2026-00078",
        "product": { "id": "0197e4a0-0000-7000-8000-0000000001007", "sku": "P-1007", "name": "ハブベアリング" },
        "quantity": 470,
        "dueDate": "2026-09-08",
        "status": "InProgress"
      }
    ]
  },
  "dueSoon": { "total": 8, "orders": [] },
  "workload": [
    { "kind": "overdue", "weekStart": null, "weekEnd": null, "orderCount": 15, "quantity": 9612 },
    { "kind": "week", "weekStart": "2026-09-22", "weekEnd": "2026-09-27", "orderCount": 6, "quantity": 3270 },
    { "kind": "later", "weekStart": null, "weekEnd": null, "orderCount": 2, "quantity": 2250 }
  ],
  "topProducts": [
    { "product": { "id": "0197e4a0-0000-7000-8000-0000000001029", "sku": "P-1029", "name": "ドアヒンジ" }, "openQuantity": 6750, "activeOrderCount": 4 }
  ],
  "completedThisWeek": { "orderCount": 3, "quantity": 1024, "from": "2026-09-21" },
  "completedThisMonth": { "orderCount": 24, "quantity": 19410, "from": "2026-09-01" },
  "onTime": { "onTimeCount": 23, "completedCount": 30, "windowStart": "2026-08-24" },
  "leadTime": { "averageDays": 13.7, "orderCount": 30, "windowStart": "2026-08-24" },
  "completionTrend": [
    { "weekStart": "2026-06-29", "weekEnd": "2026-07-05", "orderCount": 3 },
    { "weekStart": "2026-09-21", "weekEnd": "2026-09-27", "orderCount": 3 }
  ]
}

```

（`workload` は常に 10 件、`completionTrend` は 12 件。例ではそれぞれ最初と最後を示す。数値は説明用で、初期データの正確な値ではない。）

エラーコード

コード	意味	HTTP ステータス
—	未ログイン	401
—	ロールなし（クライアントは MSG-E021 を表示）	403
MSG-E013	想定外の失敗（データベース、タイムアウト）	500

400 はない：拒否する入力がない。404、409、422 もない：エンドポイントは存在するもの（何もない場合も含む）のスナップショットを読む（REQ-028 の空の状態は 0 の 200）。

2. GET /api/system/health

項目	値
説明	SCR-003 のヘルス表示（003_BD HS-01～HS-05）のために、データベースが簡単な往復に応答するかを報告する
戻り値の型	<code>SystemHealthResponse</code>
作成者／日付	Claude / 2026-09-22
最終更新者／日付	—

引数：なし。クエリ文字列は無視する。

処理概要：ダッシュボードと同じポリシーで認可し、2 秒のタイムアウトで `SELECT 1` を実行し（003_DD-FN §6）、**どちらの結果でも 200** を返す。エンドポイントが応答したこと自体がサーバーの状態で、ボディはデータベースの状態だけを持つ。このパスへのリクエストはサインインの Cookie を延長しない（DEC-019、003_DD-FN §7）。

処理フロー

ステップ	説明	呼び出し
1	認可	ポリシー <code>ProductionOrderEditor</code>
2	データベースに ping	<code>SystemHealthService.CheckAsync</code> → 003_DD-FN §6
3	<code>SystemHealthResponse</code> と <code>Cache-Control: no-store</code> 付きの 200。Cookie は再発行しない	—

リクエストの項目：該当なし。

レスポンスの項目 — `SystemHealthResponse`

No	名前	変数名	型	長さ	必須	繰り返し（配列）	値の加工	例	説明	備考
1	データベース	<code>database</code>	string	—	yes	no	<code>ok</code> <code>unavailable</code>	<code>ok</code>	何らかの理由で ping が失敗したか、2 秒より長くかかったときは <code>unavailable</code>	サーバーが報告する唯一の構成要素（DEC-020）

No	名前	変数名	型	長さ	必須	繰り返し（配列）	値の加工	例	説明	備考
2	確認日時	checkedAt	string (date-time)	—	yes	no	UTC	2026-09-22T05:05:30Z	サーバーがチェックを実行した時刻	工場時刻で表示（003_BD M-20）

それ以外は返さない：どちらの結果でも、バージョン、ホスト、接続文字列、レイテンシ、例外のテキストは返さない。

```
{ "database": "unavailable", "checkedAt": "2026-09-22T05:05:30Z" }
```

エラーコード

コード	意味	HTTP ステータス
—	未ログイン（定期確認が延長しなかったために期限切れになったセッションを含む）	401
—	ロールなし	403
MSG-E013	ping 自体以外での想定外の失敗	500 — ブラウザは「サーバー：接続不可」を表示

ping の失敗はエラーではなく database: "unavailable" の 200 である：サーバーは応答しており、それを報告するのがエンドポイントの役割である。

未解決の決定

なし。この仕様は WI-004 の DEC-006（読み取り専用）、DEC-008/DEC-009（期間、区分）、DEC-011（1 つのエンドポイント）、DEC-015（スナップショット）と、WI-002 の DEC-011/DEC-017（工場の時計）、DEC-020（CSRF）、DEC-023（RFC 9457）に基づく。ProductionOrderEditor ポリシーの再利用は、002_DD-API がすでに WI-001 DEC-015 に対して記録している制約である。